

Complete Study Notes — Theory & Q&A;

Table of Contents

Topic 1 — What Is Software Project Scheduling?

Topic 2 — The Project Scheduling Process (5 Steps)

Topic 3 — Scheduling Problems

Topic 4 — Bar Charts & Activity Network Diagrams

4A — Gantt Charts

4B — Activity Network Diagrams (CPM / PERT)

4C — Rules for Activity Network Diagrams

Bonus — Project Planning vs Project Scheduling (Student Q&A;)

TOPIC 1: What Is Software Project Scheduling?

Project Management Overview

Before scheduling, you need to understand what Software Project Management involves. Project planning has three key parts:

Part	What It Means
Project Scope Definition (WBS)	Breaking the project into smaller, manageable pieces so it can be tracked and executed.
Software Estimation	Estimating the cost and effort (person-hours) needed to complete the project.
Software Scheduling	Estimating the TIME needed and planning who does what and when. This is our main topic.

Scheduling is a PART of Planning. Planning is the bigger umbrella. Scheduling sits inside planning alongside scope definition and cost estimation.

What Is Project Scheduling? — 5 Key Activities

Your slides define project scheduling through 5 key activities. Each one is explained in detail below:

1. Split the project into tasks

You cannot manage a big project as one unit. You break it into smaller, clearly defined tasks using a Work Breakdown Structure (WBS).

■ *Example: Building a banking app → requirements gathering, database design, UI development, API development, testing, deployment. Each is a separate task.*

2. Estimate time and resources for each task

Once tasks are defined, you estimate how many days each will take, how many people are needed, and what tools or budget are required. This is an educated estimate based on experience.

■ *Example: Task: 'Build login module' → estimated 5 days, 2 developers, 1 tester.*

3. Organize tasks concurrently

Not every task depends on another. Some tasks can run in PARALLEL at the same time, making optimal use of the workforce and saving significant time.

■ *Example: While one developer writes the login module, another simultaneously works on the dashboard. No need to wait for one to finish before the other starts.*

4. Minimize task dependencies

A dependency means Task B cannot start until Task A finishes. The more dependencies, the more delays are possible. Minimize unnecessary dependencies wherever possible.

■ *Example: Testing depends on coding — unavoidable. But code review and documentation can sometimes run in parallel with coding — reducing unnecessary dependencies.*

5. Relies on project manager intuition and experience

Scheduling is not a perfect science. Two project managers given the same project might produce different schedules. A seasoned manager knows testing always takes longer than estimated and adds buffer time.

■ *Example: A junior manager estimates 3 days for a feature. A senior manager estimates 7 days knowing from experience that integration always causes delays.*

Key Exam Points

- ✓ Project scheduling splits work into tasks, estimates time/resources, and organizes them efficiently.
- ✓ Tasks should be PARALLELIZED wherever possible to save time.
- ✓ Dependencies between tasks must be MINIMIZED to avoid delays.
- ✓ Scheduling is both a science AND an art — experience plays a major role.
- ✓ Scheduling is a SUBSET of project planning — not a separate activity.

Exam Questions & Model Answers

■ Exam Q: What is project scheduling? What are its key activities?

Model Answer: Project scheduling is the process of splitting a software project into manageable tasks, estimating the time and resources required for each, and organizing those tasks to make optimal use of the available workforce. Key activities include: identifying all tasks, estimating their duration and resource requirements, organizing tasks to run concurrently wherever possible, minimizing dependencies between tasks to prevent delays, and monitoring progress throughout the project. Project scheduling relies heavily on the project manager's intuition and experience since estimation is not an exact science.

TOPIC 2: The Project Scheduling Process — 5 Steps

The 5-Step Process Diagram

Your Slide 4 shows a flow diagram with 5 boxes connected by arrows. This is the step-by-step process a project manager follows when scheduling a software project. There is also a feedback loop from Step 5 back to Step 3.

Identify Activities → Identify Possible Dependencies → Estimate Resources → Assign People to Conduct Activities → Create Activity Network and Bar Charts Feedback loop: Step 5 feeds back to Step 3 (Estimate Resources) making this ITERATIVE.

Step 1 — Identify Activities

The very first step is listing every single task that needs to be done in the project. This comes from the Work Breakdown Structure (WBS) — where you break the project into smaller and smaller pieces until each piece is a concrete, doable task.

■ *Example: Mobile app project tasks: Gather requirements, Design database schema, Build login module, Build dashboard, Write unit tests, Deploy to server, Write user documentation.*

Step 2 — Identify Possible Dependencies

Once all activities are listed, you ask: Which tasks must finish before another can begin? There are 3 types of dependencies: (1) Mandatory — physically impossible to do out of order. (2) Discretionary — a preference, not a hard rule. (3) External — depends on something outside the team such as client approval.

■ *Example: Cannot test code that has not been written yet — mandatory dependency. Waiting for client to approve requirements before coding begins — external dependency.*

Step 3 — Estimate Resources

For each activity, you estimate: how many person-days it will take, what tools are needed, and what budget is required. Resources include PEOPLE, TIME, MONEY, and TOOLS — not just money.

■ *Example: Task 'Build login module' → 2 developers, 5 days, 1 testing environment, IDE licenses.*

Step 4 — Assign People to Conduct Activities

Now you assign specific team members to specific tasks based on their skills and expertise, their availability, and the task requirements.

■ *Example: Assign your best database expert to schema design. Assign junior developers to unit testing. Never assign your only expert to two critical parallel tasks simultaneously.*

Step 5 — Create Activity Network and Bar Charts

Finally, you draw the actual schedule using Activity Network Diagrams (show task order and dependencies) and Bar Charts/Gantt Charts (show tasks plotted against calendar time). These are the OUTPUTS of the scheduling process.

■ *Example: A Gantt chart showing Week 1: Requirements, Week 2-3: Design, Week 3-4: Coding, Week 5: Testing.*

The Feedback Loop — Why It Exists

Notice in the diagram that there is an arrow going BACKWARDS from Step 5 back to Step 3. This is because when you draw the activity network, you might discover resource conflicts (two tasks needing the same person simultaneously), paths that are too long and need more resources, or estimates that were unrealistic once you see the full picture. So you go back and re-estimate resources, re-assign people, then redraw. This makes scheduling an ITERATIVE process — not a one-time straight line.

■ *Analogy: Like planning a road trip. You plan the route, realize the drive on Day 2 is 14 hours which is too long, so you go back and adjust — add a stop, split the driving differently. You iterate until the plan is realistic.*

Step	Name	Output
1	Identify Activities	Complete list of all project tasks
2	Identify Possible Dependencies	Dependency list (mandatory/discretionary/external)
3	Estimate Resources	Time, cost, and people estimates per task
4	Assign People to Activities	Team member assignments
5	Create Network and Bar Charts	Activity network diagram + Gantt chart

Key Exam Points

- ✓ The scheduling process has FIVE steps — know them in exact order.
- ✓ Step 1 uses the WBS to identify all activities.
- ✓ Step 2 identifies THREE types of dependencies: mandatory, discretionary, external.
- ✓ Resources = people + time + money + tools — not just money.
- ✓ The FEEDBACK LOOP from Step 5 back to Step 3 makes this an iterative process.
- ✓ Final output = Activity Network Diagram + Bar Charts (Gantt).

Exam Questions & Model Answers

■ Exam Q: Describe the project scheduling process with the help of a diagram.

Model Answer: The project scheduling process consists of five steps. First, all project activities are identified using the Work Breakdown Structure. Second, possible dependencies between activities are identified — these may be mandatory, discretionary, or external. Third, resources required for each activity are estimated in terms of people, time, tools, and budget. Fourth, specific team members are assigned to conduct each activity based on their skills and availability. Fifth, an activity network diagram and bar charts are created to visually represent the schedule. A feedback loop exists from the final step back to resource estimation, making the process iterative — when the network reveals resource conflicts or unrealistic estimates, the manager goes back and revises them until the schedule is feasible.

TOPIC 3: Scheduling Problems

Why Is Scheduling So Difficult?

Your Slide 5 lists 4 major problems that make software project scheduling genuinely hard. These are not just theoretical — every real software project faces these challenges.

Problem 1: Estimating Difficulty Is Hard

Software is invisible and intangible. Unlike building a wall where you can measure bricks and calculate time precisely, software complexity is very hard to measure upfront. Two tasks that look identical on paper can have completely different complexity. A feature that seems simple might require redesigning the entire database. Nobody truly knows how hard a problem is until they start solving it.

■ *Example: A project manager estimates 'add a search feature' will take 3 days. The developer discovers the database has no indexing, data is poorly structured, and search needs to work across 10 tables. It ends up taking 3 weeks.*

■ *Analogy: Asking 'how long will it take to solve this math problem?' before you have even read the problem. You genuinely do not know until you dig in.*

Problem 2: Productivity Is NOT Proportional to People

Most people think: 2 people = 2x productivity. This is completely wrong. More people means more communication channels, more time in meetings, more coordination overhead. Some tasks simply cannot be parallelized no matter how many people you add. Communication channels formula: with N people there are $N(N-1)/2$ channels. With 2 people = 1 channel. With 5 people = 10 channels. With 10 people = 45 channels.

■ *Example: 1 person can dig a hole in 1 hour. Can 60 people dig it in 1 minute? No — they would just get in each other's way. Some tasks have a logical minimum time regardless of how many people you add.*

■ *Analogy: One person can write a novel alone. Adding 10 co-authors creates 45 communication channels — constant arguments about style, plot, and voice. The book takes longer.*

Problem 3: Adding People to a Late Project Makes It Even Later

This is called BROOKS' LAW — one of the most famous laws in software engineering, stated by Fred Brooks in 'The Mythical Man-Month.' When new people join a late project: (1) New people need weeks to understand the codebase — ramp-up time. (2) Existing members must STOP work to train newcomers — training burden. (3) More people = exponentially more communication channels. (4) More developers writing code = more conflicts and merge issues.

■ *Example: A chef is running late on a complicated dish. You send 5 more chefs into the kitchen. Now the original chef must explain the recipe to all 5, they bump into each other, use the same utensils, and the dish gets even more delayed.*

■ *Analogy: Brooks' Law is NOT named in your slides but IS the exact concept described. If an exam asks 'What is Brooks' Law?' — this is the answer.*

Problem 4: The Unexpected Always Happens

No matter how carefully you plan, something will go wrong that you did not predict. Common unexpected events: key developer falls sick or resigns, third-party API changes, client changes requirements mid-project, critical bug found just before launch, hardware failures. CONTINGENCY means deliberately adding buffer time and resources into your schedule to absorb unexpected problems. Industry rule of thumb: Add 20-30% extra time to every estimate as contingency buffer.

■ *Example: When driving to the airport, you leave 2 hours early for a 45-minute drive. That extra time is your contingency for traffic, parking, or security queues.*

How the 4 Problems Create a Death Spiral

Stage	What Happens
Stage 1	Hard to estimate accurately → schedule is often wrong from the start
Stage 2	Project runs late due to poor estimation
Stage 3	Manager adds more people to catch up
Stage 4	Brooks' Law kicks in — project gets even later
Stage 5	Unexpected problems make it even worse
Stage 6	Crisis management mode — project in serious jeopardy

Key Exam Points

- ✓ Estimating software difficulty is hard because software is INTANGIBLE.
- ✓ Productivity is NOT proportional to number of people — communication overhead grows exponentially.
- ✓ Communication channels formula: $N(N-1)/2$ where N = number of people.
- ✓ BROOKS' LAW: Adding people to a late project makes it even later.
- ✓ Always add CONTINGENCY TIME in scheduling — the unexpected always happens.
- ✓ Brooks' Law is NOT named in your slides but IS the concept described in Problem 3.

Exam Questions & Model Answers

■ **Exam Q: Why is adding more people to a late software project counterproductive?**

Model Answer: Adding more people to a late project actually makes it later — a phenomenon known as Brooks' Law. This happens for three reasons. First, new team members require time to understand the existing codebase and requirements, during which they produce no output. Second, existing team members must spend time training newcomers instead of doing their own work. Third, adding people exponentially increases communication channels — with N people there are $N(N-1)/2$ channels — causing more meetings, misunderstandings, and coordination overhead. The combined effect of ramp-up time, training burden, and communication explosion means the project falls even further behind.

■ **Exam Q: What is contingency in project scheduling and why is it necessary?**

Model Answer: Contingency refers to deliberately adding extra time and resources into a project schedule as a buffer against unexpected events. It is necessary because software projects always encounter unforeseen problems such as staff illness, changing requirements, technical difficulties, or infrastructure failures. Without contingency, a single unexpected event can cause the entire schedule to collapse. Industry practice recommends adding 20-30% extra time to estimates to absorb such shocks. Good project managers treat contingency not as wasted time but as essential insurance for schedule reliability.

TOPIC 4: Bar Charts & Activity Network Diagrams

Overview — Two Graphical Tools

Your Slide 7 introduces two types of graphical notation used to represent a project schedule visually:

Tool	What It Shows	Key Strength
Gantt Chart / Bar Chart	Schedule plotted against CALENDAR TIME — when each task starts and ends.	Easy to read, great for communicating with clients and management.
Activity Network Diagram	Tasks and their DEPENDENCIES — what must be done before what. Also identifies the MINIMUM project duration via the Critical Path.	Reveals task order, dependencies, and critical activities.

Student Question answered: Gantt chart plots the ESTIMATED SCHEDULE against TIME. Activity Network represents TASKS and their RELATIONSHIPS and finds the MINIMUM PROJECT DURATION through the Critical Path. Memory tip: Gantt = WHEN. Activity Network = WHAT depends on WHAT + HOW LONG minimum.

4A — Gantt Charts (Bar Charts)

A Gantt chart is a horizontal bar chart where the vertical axis lists all tasks/activities, the horizontal axis shows calendar time (days, weeks, months), each horizontal bar represents one task with its length showing duration, and the position of the bar shows when it starts and ends.

Simple Gantt Chart Example:

Task	Week 1	Week 2	Week 3	Week 4	Week 5
Requirements	■ ■ ■ ■				
Design		■ ■ ■ ■	■ ■ ■ ■		
Coding			■ ■ ■ ■	■ ■ ■ ■	
Testing					■ ■ ■ ■

Green cells = task is active during that week

Feature	Gantt Chart
Shows timeline	■ Yes — primary strength
Shows dependencies	■ Not clearly
Shows critical path	■ No
Easy to understand	■ Very easy — suitable for clients
Best used for	Communicating schedule to team and stakeholders

4B — Activity Network Diagrams (CPM / PERT)

Your Slide 8 explains that Activity Network Diagrams model the project's activities and their relationships as a network. Two best-known techniques are used:

Tech nique	Full Name	Focus
CPM	Critical Path Method	Finds the LONGEST path through the network — determines the minimum project duration. Assumes task durations are FIXED and known.
PERT	Programme Evaluation Review Technique	Handles UNCERTAINTY in time estimates by using THREE estimates per task: optimistic, most likely, and pessimistic. Calculates a weighted average.

■■ PERT is mentioned in your slides but not solved numerically. CPM is the main focus of your examples and exam.

The Activity-on-Node (AON) Box Format:

Each task in your network is drawn as a box with 6 cells:

ES	Activity Name	EF
LS	Duration	LF

Symb ol	Full Name	Meaning
ES	Early Start	The earliest time this activity CAN start
EF	Early Finish	The earliest time this activity CAN finish (ES + Duration)
LS	Late Start	The latest time this activity CAN start WITHOUT delaying the project
LF	Late Finish	The latest time this activity CAN finish without delaying the project

Symb ol	Full Name	Meaning
Dur	Duration	How many days/weeks this activity takes to complete

4C — Rules for Activity Network Diagrams

Your Slide 9 gives 6 rules for constructing an activity network. Every rule has a specific logical reason:

Rule #	Rule	Why It Exists
1	Network flows from LEFT to RIGHT	Ensures the diagram is readable and represents time moving forward.
2	Activity cannot begin until ALL predecessors are done	Enforces real-world logic — if G depends on D and E, both must complete before G starts.
3	Arrows can cross/overlap each other	In complex networks with many dependencies, crossing arrows are unavoidable and acceptable.
4	Every activity must have a unique ID	Prevents confusion when referring to tasks in calculations and discussions.
5	Looping is NOT allowed	A loop (A→B→C→A) would mean a task depends on itself — logically impossible.
6	Every activity ID must be greater than its predecessor ID	Ensures the network flows forward numerically, preventing backwards references.

Feature	Gantt Chart	Activity Network
Shows timeline	■ Yes	■ Not directly
Shows dependencies	■ Weakly	■ Clearly
Shows critical path	■ No	■ Yes
Easy to read	■ Very easy	■ Moderate
Best used for	Communicating schedule	Analyzing dependencies and critical path

Key Exam Points

- ✓ Two graphical tools: GANTT CHARTS (time-based) and ACTIVITY NETWORKS (dependency-based).
- ✓ Two techniques for activity networks: CPM (fixed durations) and PERT (uncertain durations).

- ✓ Node box contains: ES, EF, LS, LF, Activity name, Duration.
- ✓ $ES + \text{Duration} = EF$ (forward pass formula).
- ✓ $LF - \text{Duration} = LS$ (backward pass formula).
- ✓ 6 rules for drawing activity networks — know all 6.
- ✓ NO LOOPING allowed in activity networks.
- ✓ An activity starts only when ALL predecessors are complete.
- ✓ Gantt = WHEN. Activity Network = WHAT depends on WHAT + minimum project duration.

Exam Questions & Model Answers

■ Exam Q: What are the rules for constructing an Activity Network Diagram?

Model Answer: The six rules for constructing an activity network diagram are: (1) The network must flow from left to right. (2) An activity cannot begin until all its predecessor activities have been completed. (3) Arrows representing dependencies may cross each other. (4) Every activity must have a unique identifier. (5) Looping is not allowed — no activity can depend on itself directly or indirectly. (6) Every activity must have an ID greater than its preceding activity to ensure forward flow. These rules ensure the network is logically consistent and can be used to calculate the critical path and float values.

■ Exam Q: Differentiate between a Gantt chart and an Activity Network Diagram.

Model Answer: A Gantt chart is a horizontal bar chart that plots project tasks against calendar time, showing when each task starts and ends. It is easy to read and ideal for communicating the schedule to clients and management. However, it does not clearly show task dependencies and cannot identify the critical path. An Activity Network Diagram, on the other hand, models all project activities and their dependencies as a network of nodes and arrows. It shows which tasks depend on which, identifies the critical path, and determines the minimum possible project duration. It is more complex to read but far more powerful for analysis and planning. Both tools complement each other and are used together in practice.

TOPIC BONUS: Student Q&A; — Planning vs Scheduling

Student Question: What Is the Difference Between Project Planning and Project Scheduling?

Aspect	Project Planning	Project Scheduling
Focus	WHAT to do and how much it costs	WHEN to do it and WHO does it
Output	Project plan, WBS, cost estimate	Gantt chart, activity network, timeline
Question answered	What is the scope? What resources needed?	In what order? By when? By whom?
Comes first?	■ Yes — planning comes first	Scheduling comes AFTER planning
Tools used	WBS, cost models, effort estimation	Gantt charts, CPM, PERT
Level	High level — big picture	Detailed — day by day

Memory Trick: Planning decides the DESTINATION. Scheduling decides the ROUTE and the DRIVER. **Key relationship:** Scheduling is a SUBSET of Planning. Our main topic in this chapter is PROJECT SCHEDULING specifically.

Student Question: Is Scheduling Our Main Topic or Planning?

Scheduling is a PART of Planning. Your slides are specifically about Project Scheduling — so that is our main topic. Planning is just the bigger umbrella it sits under.

Level	Topic
Bigger umbrella	Project Planning
■■■ Component 1	Scope Definition (WBS)
■■■ Component 2	Cost / Effort Estimation
■■■ Component 3 ← OUR MAIN TOPIC	Project Scheduling (Gantt, Activity Networks, CPM, PERT, Float/Slack)

End of Theory Section | Project Scheduling Study Notes | Numericals (CPM Examples) will be added after classroom solving.